

Prediction Credit Card Fraud Using Logistic Regression and Random Forests

Abstract—This paper aims to find the fraud in credit card transactions. Two algorithms were used, Logistic Regression and Random Forest. The original data set contains 300+ features. Many features are signals from the Vesta system, which are proprietary but useful parameters in finding fraudulent transactions. This report consists of a literature review with papers in the domain of credit card frauds, data processing in understanding the nature of the dataset. We present algorithms to predict fraud and present the results with evaluation metrics. This paper aims to find the fraud in credit card transactions. Two algorithms were used, Logistic Regression and Random Forest. The original data set contains 300+ features. Many features are signals from the Vesta system, which are proprietary but useful parameters in finding fraudulent transactions. This report consists of a literature review with papers in the domain of credit card frauds, data processing in understanding the nature of the dataset. We present algorithms to predict fraud and present the results with evaluation metrics.

Index Terms—Random Forest, Logistic Regression, Credit Card Fraud

I. INTRODUCTION

Fraud in the credit card industry is a prevalent problem. The Nilson Report [7] estimates that credit card fraud leads to the loss of 33.83 billion dollars annually. This figure has been on the rise over the years. Companies in the financial sector spend significant time, effort, and money on providing chargebacks for fraudulent transactions. Normally, companies are reactive and manual in the chargeback process. They first investigate merchants or transactions manually over a defined set of rules. Therefore, it is imperative to predict fraud using the available data. A credit card is omni-channel, meaning both online and offline transactions, which makes predicting fraud from a dataset tricky. It requires careful and methodical feature engineering to ensure that the fraud detection works across channels.

In this paper, we have used a Credit Card dataset from the IEEE-CIS credit card dataset created by Vesta Corporation, which is a pioneer in e-commerce payment solutions.

We consider two supervised learning algorithms, Logistic Regression and Random Forest, for predicting fraud in our dataset.

A. Objectives

- Train the model based on relevant features and use supervised learning techniques to predict fraud in credit card transactions.
- Work with imbalanced data and extensive features present in the dataset
- Evaluate based in metrics such as Precision, Recall
- Provide insights on how the dataset under various settings.

II. LITERATURE REVIEW

We have referenced 5 research papers for the literature review in the domain of credit card fraud analysis.

A. Class imbalance and Missingness patterns

The first paper titled "Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies" [1] provides data characteristics of the datasets pertaining to credit card fraud use cases.

The "IEEE-CIS Fraud Detection" [6] has severe class imbalance, with the fraud rate of just 3.5%. Hence, the traditional accuracy metrics are not useful evaluation metric for detecting fraud. As the formula consists of **True Positive** and **True Negative**, a classifier that always predicts **Not fraud** achieves very high accuracy. Instead, other metrics such as **precision** and **recall** are used, as they focus on the **True Positive** data points.

B. Practical implementation of Credit Card Fraud in the real world

The paper titled "A Data Mining Based System For Transaction Fraud Detection" [4] provides practical deployment of random forest in a real-world setting. Instead of following the traditional theoretical part of finding the best evaluation metrics, they propose a

semi-automatic framework wherein they route transactions based on a random classifier. If they find a transaction as high-risk, they add manual intervention to make sure that an expert can look into it on a case-by-case basis. The expert then decides whether the transaction is high-risk. This paper highlights the importance of deploying models in the real world and the risk associated with following the theoretical model just as it is.

C. Choosing the right features

We analyze the paper "HOBAs: A novel feature engineering methodology for credit card fraud detection" [3] for how feature engineering suits this dataset.

- **Categorical Imputation:** For categorical values with more than 5% missing values, they added "missing" categories to retain the information.
- **Numerical imputation:** Missing numerical values were replaced with mean values, grouped by fraud and legitimate transaction groups.
- **Feature removal:** They removed entire features which were highly correlated and features with low information gain.

D. Choosing the algorithm

We have seen the paper "Customer Transaction Fraud Detection" [5] for helping decide on the techniques to be used in this paper for the Credit Card Fraud use case.

Here Logistic Regression was used as a baseline for fraud classification. Random forest techniques were mentioned, and how they work for this dataset. In this paper, the nodes were split based on Gain Ratio and Gini Index. Evaluation metrics such as AUC-ROC and accuracy were better in Random Forest than in Logistic Regression.

E. Improving Random Forest

We have referenced this paper "A Machine Learning Approach for Credit Card Fraud Detection in Massive Datasets Using SMOTE and Random Sampling" to help improve the recall score in Random Forest. The author found that while SMOTE improved the Recall evaluation metric, it led to drastically increasing "False Positive" data. To prevent this, it proposed using an improved version of SMOTE, **SMOTE-ENN**.

Based on the literature, we have chosen the data pre-processing strategy, techniques used, and evaluation metrics incorporated to compare performance of different techniques.

III. DATA MANAGEMENT

A. Dataset Description

The dataset was fetched from Kaggle and IEEE, named "IEEE-CIS Fraud Detection" [6].

Dataset contains 2 tables: **Transaction Table** and **Identity table**.

- **Transaction table:** Contains general transaction information such as transaction amount, date, card details, Vesta specific features (V1-V339), masked indicators such as names, email domains.

TABLE I. Transaction Table Features

Feature Name	Description
TransactionDT	Transaction Date
TransactionAmt	Transaction Amount
ProductCD	Product code for the transaction
card1 – card6	Payment card info (e.g., card type, bank ID)
addr1, addr2	Address identifiers
dist1, dist2	Distance features
P_emaildomain	Purchaser email domain
R_emaildomain	Recipient email domain
M1 – M9	Masked indicators (e.g., names, addresses)
C1 – C9	Counting features
D1 – D15	Timedelta features
V1 – V339	Vesta engineered features

- **Identity table:** Contains additional information such as network information, browser type.

TABLE II. Identity Table Features

Feature Name	Description
TransactionID	Transaction identifier linking to transaction table
DeviceType	Device type information
DeviceInfo	Device type information
id_01, id_02	Identity distance and score features
id_03, id_04	Domain reputation scores
id_05, id_06	Device registration and mismatch indicators
id_07, id_08	Time-based features
id_09, id_10	Device count features
id_11	Device reputation score
id_12, id_15, id_16	Found / NotFound status
id_13, id_14	Regional distance features
id_17 – id_22	Identity score and registration features
id_23	IP proxy indicator (categorical)
id_24, id_25, id_26	Numeric identity features
id_27, id_28, id_29	Match status indicators (New, Found)
id_30, id_31, id_32	OS and browser information
id_33, id_34	Device screen properties
id_35 ... id_38	Boolean matching indicators

TABLE III. Sample of the Transaction Data

TxID	TxDt	TxAmt	Prod	cd1	cd2	cd3	cd4	cd5	cd6	a1	a2	d1	P_email	C1 ...	D1 ...	M1	V1 ...
3663549	18403224	31.95	W	10409	111	150	visa	226	debit	170	87	1	gmail.com	6.0 ...	419 ...	T	1.0 ...
3663550	18403263	49.00	W	4272	111	150	visa	226	debit	299	87	4	aol.com	3.0 ...	12 ...	T	1.0 ...
3663551	18403310	171.00	W	4476	574	150	visa	226	debit	472	87	2635	hotmail.com	2.0 ...	22 ...	T	1.0 ...
3663551	18403310	171.00	W	4476	574	150	visa	226	debit	472	87	2635	hotmail.com	2.0 ...	22 ...	T	1.0 ...
3663551	18403310	171.00	W	4476	574	150	visa	226	debit	472	87	2635	hotmail.com	2.0 ...	22 ...	T	1.0 ...

TABLE IV. Sample of the Identity Data

TxID	id_01	id_02	id_05	id_06	id_11	id_15	id_16	id_17	id_28	id_29	id_35	Device	Info
2987004	0.0	70787	...	...	100.0	New	NotFound	166	New	NotFound	T	mobile	SAMSUNG SM-G892A
2987008	-5.0	98945	0.0	-5.0	100.0	New	NotFound	166	New	NotFound	T	mobile	iOS Device
2987010	-5.0	191631	0.0	0.0	100.0	Found	Found	121	Found	Found	F	desktop	Windows
2987011	-5.0	221832	0.0	-6.0	100.0	New	NotFound	225	New	NotFound	F	desktop	...
2987016	0.0	7460	1.0	0.0	100.0	Found	Found	166	Found	Found	T	desktop	MacOS

B. Data Preprocessing

- **Handling missingness:**
 - Drop columns with >90% missing values.
 - Impute **categorical values** with mode and **numeric values** with mean.
- Do composite features on the following columns
 - **C1_C2_sum**
 - **D10_D15_diff**
 - **card1_card2_sum**
- Encode **categorical** features such as Product Code into integers
- **Feature engineering for TransactionDT:** As fraud behaviour is time-dependent, we split it into day, month and year; after which we do cyclical encoding for hour and day-of-week. We also split based on month.
- **Log transform for skewed variables:** Distances and fraud amounts are long-tailed. Logging in to these features helps to stabilize the scale of these features.

C. Dimension reduction

- **Feature selection methods:** Score all features on **chi**, **mutual information**, **ExtraTrees** and **F Classification** and select the top 80 features. We notice **V257** comes in top in **F Classification** and **Mutual Information Scores**. For **Chi2** and **ExtraTrees Classifier**, **card6** and **Transaction Date** are the top features respectively. As we do not have any common features in most of the methods, we cannot conclude which feature we can reduce. A better option would be to use PCA.

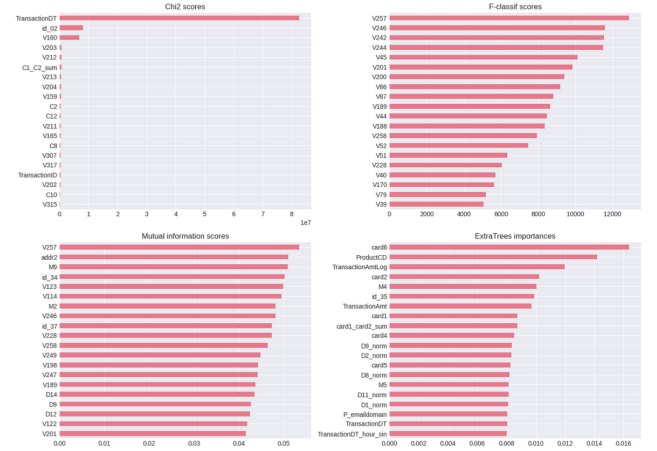


Fig. 2. Comparative Feature Importance Analysis Across Selection Methods

- **PCA:** We use Scikit's *PCA* function to reduce number of features. As we have large number of features, we take the top 80 features.

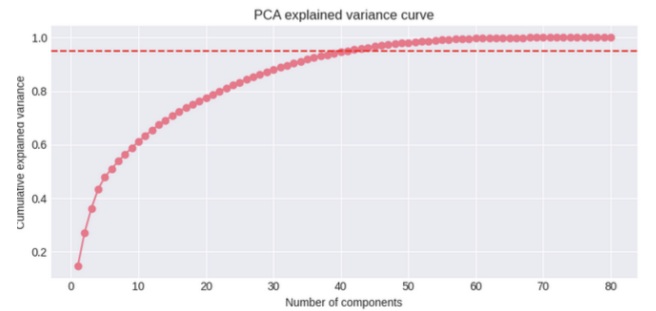


Fig. 3. PCA Variance

- **Correlation matrix**
The dataset has very large number of features. Typical correlation measures only *linear relationships*. Fraud dataset are highly complex with non-

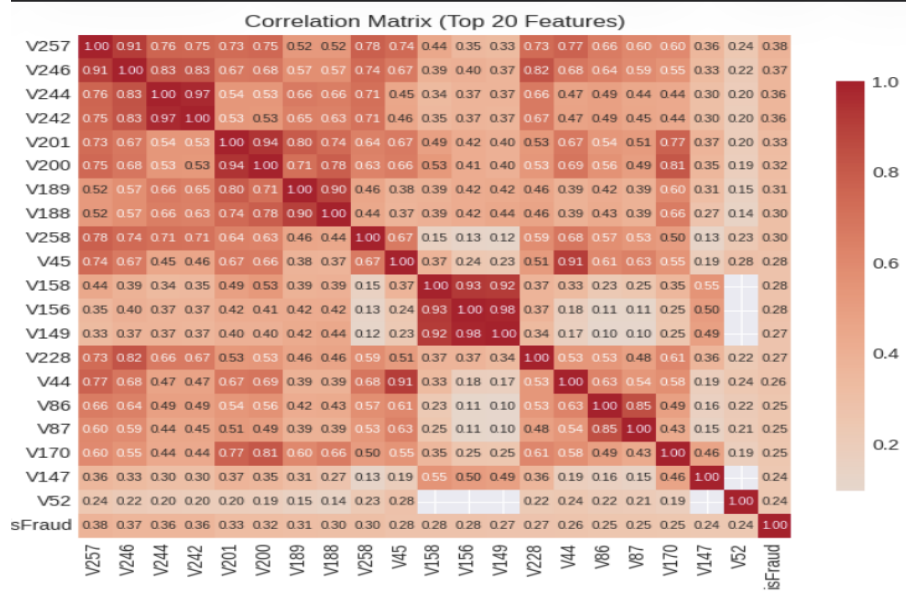


Fig. 1. Correlation Matrix

linear high dimension dataset. A feature might have zero correlation with the target Fraud variable but can be very high correlation when added with another feature. For example, combining transaction amount with time of day.

IV. METHODOLOGIES

This paper takes the supervised learning approach to determine the target variable *isFraud*. Within the supervised learning framework, there are two kinds of outcomes. Firstly, there is **classification** which is when the target variable is discrete. Secondly, it is **regression**, where the output is a continuous variable.

Since *isFraud* is a discrete *Yes* or *No*, we have chosen the **classification** classes of algorithms.

A. Logistic Regression

Logistic Regression is a binary classification regression methodology. It provides a probability value between 0 and 1 for the fraudulent transaction.

$$P(\text{isFraud} = 1|X) = \frac{1}{1 + e^{-z}}$$

1) *Logit equation* (z): This is where the features of our dataset are added. It is of the form:

$$z = \beta_0 + \beta_1(\text{TransactionAmt}) + \beta_2(\text{dist1}) + \sum_{i=1}^{14} \beta_{C_i}(C_i) + \dots$$

where β_0 is bias

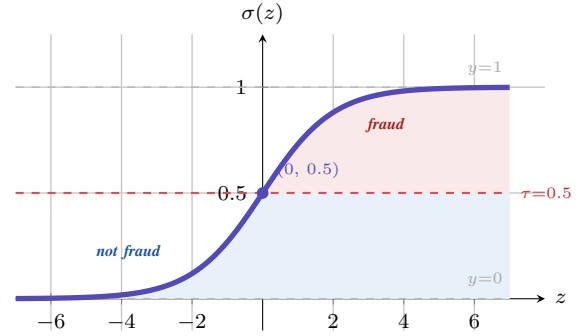


Fig. 4. Sigmoid where red = fraud region; blue = legitimate.

2) *Objective function*: β_0 weights are calculate based on the **Objective function**, namely **Binary Cross Entropy** which is of the form:

$$J = -\frac{1}{N} \sum_i \left[\underbrace{f_i \log(p_i)}_{\substack{\uparrow \\ \text{Activates if } f_i=1 \\ \text{(Fraud)}}} + \underbrace{(1-f_i) \log(1-p_i)}_{\substack{\uparrow \\ \text{Activates if } f_i=0 \\ \text{(Legitimate)}}} \right]$$

This function is to be minimized via Gradient descent.

- $J(\beta)$: Error cost function which is to be minimized.
- $y^{(i)}$: Ground truth from the dataset for the i -th transaction.

- $h_{\beta}(x^{(i)})$: Probability predicted from the sigmoid function.

Justification:

Logistic Regression takes in the weights to the features that are in the data to provide an output based on the sigmoid function.

We use Logistic regression as a baseline for other method (Random forest).

B. Random Forest

Random forest is an enhancement of decision trees.

At its core, it samples the input data set into various samples. These samples are then used to create a decision tree of their own. Lastly, all the decision trees are aggregated to get the final prediction.

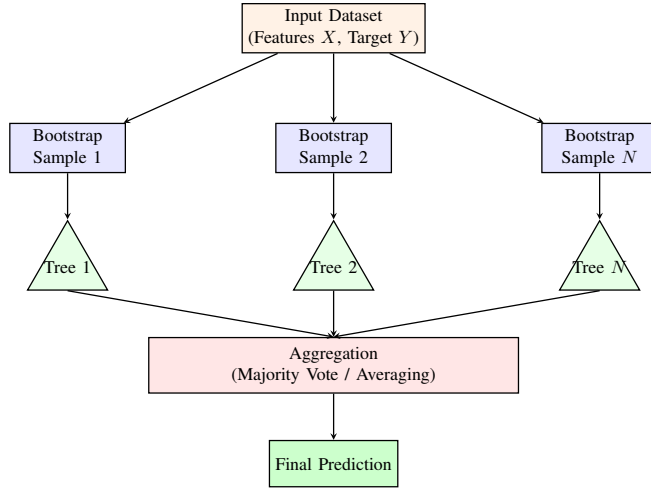


Fig. 5. Conceptual Architecture of the Random Forest Model

Justification

For this data set with high dimensional (300+ features) which are imbalanced, random forest is effective as it does feature selection implicitly using random subsampling.

Since it uses random sampling, it overcomes the issue of overfitting that arises in decision with high number of features. Majority voting of multiple trees ensures that the model aggregates to an output that is the most popular.

We have seen recall value drop when using Random Forest. This is due to high imbalance in the dataset. We have extended Random forest to use **SMOTE**. The SMOTE enhances Random forest's tendency to ignore rare instances as noise by forcing the decision trees to expand their decision boundaries.

We have used **imbalanced learn** [9] library to incorporate SMOTE in our notebook.

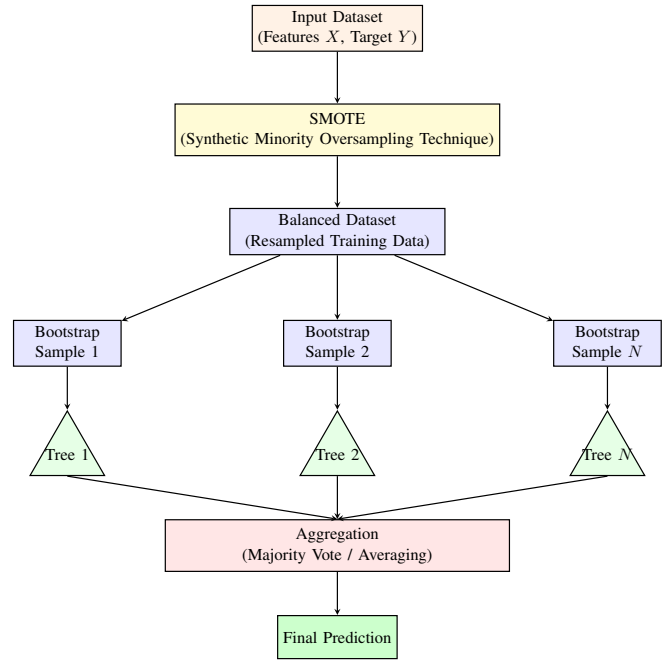


Fig. 6. Conceptual Architecture of the Random Forest Model with SMOTE

V. FINDINGS AND RESULTS

Since the dataset was non-trivial, both in terms of data size and number of features, the average time for running the experiment was approximately 1 hour.

VI. CONFIGURATION

Scikit's [8] method were used with giving specific hyperparameters.

1) : Logistic Regression

```
1 LogisticRegression(
2     C=best_c,
3     max_iter=500, class_weight="balanced",
4     solver="liblinear",
5     random_state=RANDOM_STATE,
6 )
```

Listing 1: LogisticRegression configuration

2) : Random Forest

```
1 RandomForestClassifier(
2     n_estimators=600,
3     max_depth=18,
4     min_samples_leaf=6,
5     max_features="sqrt",
6     class_weight=class_weight_dict,
7     n_jobs=-1,
8     random_state=RANDOM_STATE
9 )
```


Listing 2: RandomForestClassifier configuration

A. Evaluation Metrics

		PREDICTED FRAUD	PREDICTED LEGITIMATE
ACTUAL FRAUD	ACTUAL FRAUD	TRUE NEGATIVE (TP) Legitimate correctly identified	FALSE NEGATIVE (FN) Fraud missed (Type II error)
	ACTUAL LEGITIMATE	FALSE POSITIVE (FP) Legitimate flagged (Type I error)	TRUE POSITIVE (TP) Fraud correctly identified

Fig. 7. Confusion Matrix for Fraud Detection

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

$$F_1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN} \quad (5)$$

$$\text{AUC} = \sum_{i=1}^{n-1} \frac{(FPR_{i+1} - FPR_i)(TPR_i + TPR_{i+1})}{2} \quad (6)$$

- **Accuracy:** This is high in Random forest. Usually, accuracy is not the best metric for success, as it can very high due to an imbalanced dataset for "Non Fraud" data points whereas actual fraud may not be predicted accurately.
- **Precision:** Random forest scores normally whilst Logistic Regression scores very poorly.
- **Recall:** Random forest scores poorly whilst Logistic Regression scores well.

- **F1 Score:** Random forest scores normally whilst Logistic Regression scores very poorly.
- **ROC-AUC:** Random Forest and Logistic Regression are similar in terms of this metric.

TABLE V. Model Performance Metrics

Model	Accuracy	F1	Precision	Recall	ROC-AUC
Logistic Regression	0.634	0.128	0.070	0.807	0.797
Logistic Regression (PCA 41)	0.634	0.126	0.069	0.786	0.793
Random Forest (Base)	0.968	0.462	0.530	0.410	0.866
RF (SMOTE Full)	0.967	0.477	0.514	0.444	0.881
RF (SMOTE Partial 0.5)	0.969	0.503	0.547	0.470	0.879
RF (SMOTE-ENN)	0.968	0.464	0.540	0.408	0.878
RF (SMOTE-Tomek)	0.971	0.486	0.597	0.410	0.883

B. Confusion Matrix

Confusion Matrix - Logistic Regression

Actual \ Predicted	Not Fraud	Fraud
Not Fraud	17190	10164
Fraud	189	763

(a) Logistic Regression

Confusion Matrix - Logistic Regression (PCA 41)

Actual \ Predicted	Not Fraud	Fraud
Not Fraud	17209	10145
Fraud	204	748

(b) Logistic Regression with PCA

Confusion Matrix - Random Forest

Actual \ Predicted	Not Fraud	Fraud
Not Fraud	27008	346
Fraud	562	390

(c) Random Forest

Confusion Matrix - RF (SMOTE-Tomek)

Actual \ Predicted	Not Fraud	Fraud
Not Fraud	27091	263
Fraud	562	390

(d) Random Forest with SMOTE

Fig. 8. Confusion matrices for all four models

The above figure shows the confusion matrix for all the techniques along with the different settings.

The bottom right and top left are correct predictions of fraud. We can see Random Forest outperform Logistic Regression in True Positives.

C. Discussion

We have done two settings for each technique. For Logistic Regression, we did it with all features and one with feature reduction using PCA.

For Random Forest, we have done a vanilla Random Forest and also Random Forest with SMOTE enhancement. SMOTE was further configured with **SMOTE-full**, **SMOTE-partial 0.5**, **SMOTE-ENN** and **SMOTE-Tomek** settings.

Based on the metrics, we can see that the recall is not optimal for either Random Forest. This is due to the large, imbalanced dataset. Logistic regression is good for fast training and recall.

Random forest is good in terms of precision and accuracy.

Random forest with SMOTE partial 0.5 setting has the best F1 score.

VII. CONCLUDING REMARKS

This project aimed at taking a real-world problem, such as fraud in credit card transactions and finding a technique to predict the fraud correctly.

A. Achievements

We have used **logistic regression** and **random forest** in different settings for predicting fraud in a credit card transaction dataset.

The dataset was non-trivial, both in terms of dataset size and feature size. *Transaction table* had **590,540 rows and 394 features** while *Identity table* had **144,233 rows and 41 features**. PCA helped reduce the number of principal components to 41, thereby considerably reducing the feature space.

We have found that Random Forest had provided better prediction than Logistic Regression, in terms of precision.

We utilized SMOTE in various settings for better sampling while building decision trees and eventually Random Forests.

B. Technical challenges

Due to the dataset's large size, running the notebook end-to-end took around 1 hour. Also, selecting hyperparameters for Random Forest was difficult. To overcome this, SMOTE as a configuration with random state.

Data Preprocessing was challenging because temporal feature manipulation was required. Aggregating

features was also a challenge, as there were a lot of anonymized features.

C. Future enhancements

For future work, other techniques can be employed to improve training time and recall. Random Forests are computationally expensive to tune given the high number of features and data.

LightGBM provides a new learning paradigm known as Bagging to train decision trees independently. It also uses concept of **Gradient Boosting** to build shallow trees sequentially and leaf-wise instead of depth-wise generation in Random Forest. This paper [10] highlights improvements of using LightGBM over random forests and other algorithms.

REFERENCES

- [1] Hayat, K.; Magnier, B. Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies. *Mathematics* 2025, 13, 2563. <https://doi.org/10.3390/math13162563>
- [2] Siddhartha Bhattacharyya, Sanjeev Jha, Kurian Tharakunnel, J. Christopher Westland, Data mining for credit card fraud: A comparative study, *Decision Support Systems*, Volume 50, Issue 3, 2011, Pages 602-613, ISSN 0167-9236, <https://doi.org/10.1016/j.dss.2010.08.008>.
- [3] Xinwei Zhang, Yaoci Han, Wei Xu, Qili Wang, HOBA: A novel feature engineering methodology for credit card fraud detection with a deep learning architecture, *Information Sciences*, Volume 557, 2021, Pages 302-316, ISSN 0020-0255, <https://doi.org/10.1016/j.ins.2019.05.023>.
- [4] W. Deng, Z. Huang, J. Zhang and J. Xu, "A Data Mining Based System For Transaction Fraud Detection," 2021 IEEE International Conference on Consumer Electronics and Computer Engineering (ICCECE), Guangzhou, China, 2021, pp. 542-545, doi: 10.1109/ICCECE51280.2021.9342376.
- [5] Y. Zhang, J. Tong, Z. Wang and F. Gao, "Customer Transaction Fraud Detection Using Xgboost Model," 2020 International Conference on Computer Engineering and Application (ICCEA), Guangzhou, China, 2020, pp. 554-558, doi: 10.1109/ICCEA50009.2020.00122.
- [6] Yuqun Cao, "IEEE-CIS Fraud Detection", IEEE Dataport, March 12, 2026, doi:10.21227/y5e7-wp63
- [7] Nilson Report, "Card Fraud Losses Worldwide in 2023," Nilson Report, Dec. 2024. [Online]. Available: <https://nilsonreport.com/articles/card-fraud-losses-worldwide-in-2023/>. [Accessed: Mar. 15, 2026].
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [9] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1-5, 2017.
- [10] Huang, K. (2020). An Optimized LightGBM Model for Fraud Detection. *Journal of Physics: Conference Series*. 1651. 012111. 10.1088/1742-6596/1651/1/012111.